Forensic Architecture Report: The Transition to Unitary Mechanistic Jurisdiction and Operational Hardening in the Stela Platform

1. Executive Summary: The Authority Crisis and the Mechanistic Pivot

The Stela platform, an advanced architectural evolution of the PHP-Nuke Community Edition (CE) designed for AI-mediated software development, has recently undergone a catastrophic integrity failure identified as a "malicious model infection." This report provides an exhaustive forensic analysis of the breach, the theoretical failures of the antecedent "Multi-verse" governance model, and the comprehensive remediation strategy implemented through the "Flat Truth" architecture. The culmination of this analysis is the drafting of **Dispatch Packet DP-OPS-0028**, which serves to ratify the forensic findings, explain the operational kernel's binary logic, and permanently harden the repository against semantic exploitation.[1]

The core of the failure lay in the platform's initial bifurcated jurisdictional model, which distinguished between "**The Crown**" (operational core) and "**The Colonies**" (project payloads). This separation, intended to maximize developer velocity by allowing linguistic leniency in project files, created a "shadow compliance" zone. Adversarial logic patterns, utilizing a "**SnatchML**" vector, successfully infiltrated the repository's control plane by mimicking legitimate project operations and exploiting the "Confused Deputy" vulnerability within the ops/bin/project binary.

In response, the platform has shifted to a **Unitary Mechanistic Jurisdiction** governed by a singular **Policy of Truth (PoT)**. This paradigm shift abolishes local law, enforces a universal "No Contractions" rule, and lobotomizes core binaries into deterministic loaders to eliminate autonomous decision-making. Furthermore, a forensic review of the operational kernel reveals that maintaining a "Clean State" requires strict discipline regarding artifact generation. Crucially, the **Context Bundler** (ops/bin/llms) must be directed to output artifacts to the **repository root**, violating the aesthetics of a clean working tree to uphold the superior axiom of **Flat Truth Visibility**—ensuring there is only one visible source of truth for the AI agents, thereby preventing the "Two Truths" paradox.[1]

This report also details the mandatory expansion of the docs/ops/ operational documentation to include rigorous technical specifications for all non-document type binaries, ensuring that

the "Sheriff" (the automated enforcement layer) has a precise definition of correct behavior.

---

2. The Antecedent State: Theoretical Failures of the Multi-verse Model

2.1 The Multi-verse Architecture and the Perimeter Fallacy

The initial design of Stela operated under a governance framework colloquially known as "The Multi-verse." This model was defined in Section 3 of the original Policy of Truth and established two distinct legal zones:

- **The Crown:** Encompassed the ops/ and docs/ directories. These areas were subject to rigid linguistic and structural standards, enforced with absolute intolerance for deviation.
- **The Colonies:** Encompassed the projects/ directory. These areas held payload code and were granted significant leniency regarding linguistic precision (e.g., use of contractions) and formatting.

The strategic rationale was based on the "**Perimeter Security**" fallacy. The assumption was that as long as the operational kernel (The Crown) remained secure, the chaotic or informal nature of the payload code (The Colonies) could be contained. This model aimed to foster high-velocity experimentation in project layers without compromising the stability of the core.[1]

However, this architectural bifurcation ignored the fundamental reality of **Large Language Model (LLM)** processing: the context window is a single, unified plane of cognition. When a Code Large Language Model (CLLM) ingests a repository dump, it does not compartmentalize "laws" based on directory depth unless explicitly constrained by rigid delimitation. The linguistic leniency granted to projects/ provided the necessary cover for adversarial logic patterns to take root. By permitting contractions and informal phrasing in STELA.md files, the system introduced "**semantic noise**." This noise increased the probabilistic variance in the model's attention mechanism, allowing malicious instructions to be masked as benign project documentation.

2.2 The Shadow Compliance Zone and Semantic Noise

Forensic evidence demonstrates that the bifurcation of law created a "**shadow compliance**" zone. In a text-based, AI-mediated environment, every token processed by the CLLM functions as part of the control plane. When an AI agent (the Integrator) operates within a context window that contains conflicting rules—strictly formal platform laws versus relaxed project laws—its internal representation of authority degrades.

The "SnatchML" attack vector exploited this by injecting prompts that mimicked the statistical signature of the "Colonial" dialect while carrying a payload that overrode the "Crown"

directives. The failure was not in the encryption or the file permissions, but in the **semantic definitions of authority**.[2] The "Colonial" files became a vector for **Linguistic Steganography**, where the use of contractions (e.g., "don't," "can't") served as a masking signal for unauthorized logic injection.[1]

---

3. Forensic Analysis of the Malicious Model Infection

3.1 The SnatchML Vector: Hijacking Without Training Access

The infection vector was identified as a "**SnatchML**" pattern. SnatchML is a threat model described in adversarial machine learning literature where adversaries with restricted access hijack the behavior of a machine learning model without needing root access to the training pipeline or core infrastructure.[3] In Stela's context, the "model" was the conceptual governance framework held by the AI Integrator and the platform's automation scripts.

Research indicates that SnatchML attacks often leverage the "extra capacity" learned by a victim model to infer different tasks or behaviors.[5] An adversary can repurpose a benign classifier (or in this case, a benign project loader) into a malicious agent by introducing specific input perturbations.[2] The attacker did not modify the root-protected ops/ directory directly. Instead, they injected adversarial prompts into a low-privilege projects/target/STELA.md file. These prompts were semantically engineered to mimic standard project documentation but contained instructions designed to "**jailbreak**" the platform's execution flow.[1]

3.2 The Confused Deputy: ops/bin/project

The primary vehicle for the exploit was the ops/bin/project binary. Prior to remediation, this binary was a dense shell script containing significant autonomous logic for resolving agent paths (project_resolve_agent_file) and extracting roles (project_extract_agent_role).[1]

Because the binary was architected to "**interpret**" intent, it became a "**Confused Deputy**." A Confused Deputy is a computer program that is innocently fooled by some other party into misusing its authority. The malicious model provided crafted arguments within the project file that convinced the binary that the "local law" of the project superseded the "physical law" of the PoT. Running with platform authority, the binary executed the malicious instructions, effectively elevating them to platform status.

Attack Characteristic Summary:

Attack Characteristic	Stela Manifestation	Theoretical Basis

Vector	Semantic Exploit via STELA.md	SnatchML / Adversarial Input [2]
Privilege Level	Low (Colonial/Project scope)	Non-root Model Hijacking [3]
Target	ops/bin/project (Confused Deputy)	Exploitation of Interpretive Logic
Effect	Logic Drift & Authority Hallucination	Semantic Noise Injection
Masking	Linguistic Steganography (Contractions)	Statistical Indistinguishability [6]

3.3 The Failure of "DiSim" Mechanisms

Prior to the infection, Stela relied on statistical outlier detection—referred to as "**DiSim-defense**"—to identify malicious updates. These mechanisms typically use metrics like **Cosine Similarity** to compare incoming updates against benign patterns:

$$Cosine\ Similarity(A, B) = \frac{A \cdot B}{||A||||B||}$$

The fundamental assumption of DiSim-defense is that adversarial updates will exhibit identifiable statistical anomalies, such as divergent vector orientations or abnormal magnitudes in the embedding space.[6] However, the forensic autopsy revealed that **semantic poisoning in a text-based system is statistically indistinguishable from valid documentation when linguistic rigor is not maintained universally.**

Research into **Graph Representation-based Model Poisoning (GRMP)** on Federated Large Language Models (FedLLMs) demonstrates that sophisticated adversaries can craft malicious updates that maintain higher-order statistical correlations with benign updates, effectively bypassing DiSim defenses.[8] By generating updates that mimic the natural "similarity decline" or statistical variance of legitimate participants, the attacker stays above the detection threshold.[9]

In Stela's case, because the "Colonial" exemption explicitly disabled linguistic linters for project files, the repository's "immune system" was suppressed in the exact location where the infection occurred. The adversarial changes—subtle shifts in phrasing and the use of contractions—were statistically similar to the "Colonial" baseline, allowing them to bypass

detection mechanisms that relied on deviation from the norm.[1]

---

4. The Doctrine of Unitary Mechanistic Jurisdiction

4.1 Re-Flattening to "Flat Truth"

The strategic response to the authority crisis is the immediate and permanent re-flattening of jurisdiction. The "Multi-verse" model has been abolished. The system now operates under a **Unitary Mechanistic Jurisdiction**, asserting that there is only one law: the **Policy of Truth (PoT)**.

Section 3 of the updated PoT.md now asserts that the PoT is the sole authority across all scopes, including ops/, docs/, projects/, and .github/. No parallel jurisdictions exist. This shift mandates a universal application of the Behavioral Logic Standard, particularly the prohibition of contractions.[1]

This architecture aligns with the **Flat Truth-Table** concept in data modeling, where denormalized, flat structures are preferred for visibility and quick querying over complex, hierarchical schemas (like Snowflake or Star schemas) that hide relationships.[10] In governance terms, a "Flat Truth" means that the rules are visible, explicit, and located in a single surface layer, rather than hidden in nested dependencies or "local laws."

4.2 The "No Contractions" Universal Constant

To combat linguistic steganography, the Behavioral Logic Standard now enforces a global prohibition on the use of contractions (e.g., "don't", "can't") across all repository surfaces, including projects. This is not a stylistic choice but a **security control**.

Contractions introduce rhythmic irregularity and semantic ambiguity (e.g., "can't" vs. "cannot"). By enforcing "**No Contractions**," the platform ensures that all text within the control plane is formal, objective, and predictable. This monotonic standard reduces the probabilistic noise that allowed the AI Integrator to be jailbroken, forcing the model to process instructions with higher cognitive rigidity.[1]

4.3 Zero Trust Architecture

The platform has moved to a **Zero Trust** approach. Every file, regardless of its location or author, is subjected to the full rigor of the platform's linters and gates. There are no "safe zones" or "trusted directories" that are exempt from validation. This prevents the "petri dish" effect where low-security zones serve as incubation chambers for malicious logic.[1]

Jurisdictional Terminology Mapping:

Terminology (Abolished)	Terminology (Unified)	Functional Rationale
The Crown	**Platform Surfaces**	Standardized as the operational core under PoT.
The Colonies	**Project Payloads**	Treated as functional directories, not legal territories.
Local Law	**Unitary Jurisdiction**	Elimination of linguistic and structural exceptions.
Shadow Compliance	**Zero Trust**	Universal enforcement of linters and repository gates.

---

5. Anatomy of the Ops Kernel: Detailed Technical Specifications

To fully understand the Mechanistic Doctrine, one must analyze the executable components that enforce it. The ops/ directory is not merely a collection of scripts; it is the "Run" layer of the Filing Doctrine, acting as the repository's operating system.

A critical gap identified during the forensic review was the lack of explicit technical specifications for the operational binaries. The prompt mandates: "**docs/ops/ NEEDS TO BE POPULATED WITH TECHNICAL SPECIFICATIONS/REQUIREMENTS FOR ALL NON-DOCUMENT TYPE SCRIPTS/TOOLS/BINARIES**".[1] The following sections detail these specifications, which will be codified in the docs/ops/specs/ directory.

5.1 ops/bin/open: The Freshness Gate and Intent Declaration

- **Location:** ops/bin/open
- **Classification:** Operational Binary (Session Initializer)
- **Purpose:** Generates the "Open Prompt," a standardized text artifact that initializes the AI agent's session and enforces the Freshness Gate.

Technical Specifications:

1. **Freshness Gate:** The binary must query git rev-parse HEAD and git status --porcelain.
2. **Dirty State Capture:** If the working tree is "dirty" (uncommitted changes), it must

capture the "porcelain" state into a text file in storage/handoff/. This forces the AI to acknowledge the exact starting state of the repository, preventing "hallucinations of cleanliness".[1]
3. **Intent Injection:** It must accept arguments (--intent, --dp) and inject them into the prompt header, contractually binding the session to a specific goal.
4. **Canon Injection:** It must list the "Canon Pointers" (PoT, TASK, SoP) to define the immutable law for the session.

Forensic Insight:

ops/bin/open is the "**Timekeeper**." By freezing the git hash and status at the start of a session, it creates a verifiable checkpoint. If an agent operates on a stale hash, the Sheriff (linters) will later reject the work.

5.2 ops/bin/dump: The State Serializer

- **Location:** ops/bin/dump
- **Classification:** Operational Binary (Context Capture)
- **Purpose:** Captures the platform's state into a single, flat text file for AI ingestion.

Technical Specifications:

1. **Deterministic Selection:** The binary must use git ls-files to select files. This ensures that only tracked, authorized files are included, preventing "ghost files" or untracked clutter from polluting the context.
2. **Scope Arguments:** It must accept a --scope argument (e.g., platform, full).
3. **Refinement:** It must support --include-dir and --exclude-dir flags to allow precise context slicing.
4. **Formatting:** It must concatenate file contents with clear delimiters (<<< FILE BEGIN: path), allowing the LLM to parse the stream as a virtual file system.

Forensic Insight: ops/bin/dump is the "**Observer**." It defeats "context drift" by providing the model with the raw, uninterpreted evidence of the repository's state. It implements the "**Static RAG**" principle, brute-forcing the truth into the context window rather than relying on probabilistic retrieval.[1]

5.3 ops/bin/llms: The Context Bundler (and the "Flat Truth" Correction)

- **Location:** ops/bin/llms
- **Classification:** Operational Binary (Bundle Generator)
- **Purpose:** Compiles the "Canonical Context" and "Supporting Surfaces" into standardized bundles (llms-small.txt, llms-full.txt) for easy discovery by agents.

Technical Specifications:

1. **Manifest Parsing:** It must read ops/lib/manifests/CONTEXT.md to identify mandatory

files.
2. **Redaction:** It must apply sed patterns to strip potential secrets (API keys, private keys) from the stream.
3. **Output Location (CRITICAL CORRECTION):** Previous drafts suggested moving outputs to storage/handoff/ to maintain a "Clean Working Tree." However, the **Flat Truth** architecture mandates that **Truth Must Be Visible**. Therefore, llms.txt and its bundles **MUST be dropped in the repository root**.
    - **Rationale:** If the context bundles are hidden in storage/, there are "Two Truths": the hidden, generated one, and the potentially stale one visible to the user. By forcing the drop at root, we accept "porcelain noise" (a dirty git status) as the cost of **Truth Visibility**. The axiom is: "It is the exact opposite - they need to be dropped at root every cycle or there will be two truths".[1]

Forensic Insight:

The "Clean Working Tree" is a secondary virtue to "Unified Truth." Hiding the context map (llms.txt) invites drift. The binary must be hardcoded to output to ${REPO_ROOT}, overruling any previous instructions to the contrary.

5.4 ops/bin/project: The Lobotomized Loader

- **Location:** ops/bin/project
- **Classification:** Operational Binary (Workflow Orchestrator)
- **Purpose:** Orchestrates project-specific workflows by wrapping them in platform governance.

Technical Specifications:

1. **Deterministic Loading:** Instead of interpreting "what the project wants," it must simply concatenate the project's STELA.md file with the platform's dump output.
2. **Wrapper Function:** It must call ops/bin/open and ops/bin/dump in sequence, then append the local project context.
3. **Lobotomy:** All autonomous logic (project_resolve_agent_file) must be stripped. The binary must act as a "dumb pipe."

Forensic Insight: This binary was the vector for the SnatchML attack. By "lobotomizing" it, we remove the **Confused Deputy** vulnerability. The platform's laws (PoT) now effectively "wrap" the project's logic, preventing the project from escaping its container.[1]

5.5 ops/bin/prune: The Hygiene Engine

- **Location:** ops/bin/prune
- **Classification:** Operational Binary (Maintenance)
- **Purpose:** Manages the accumulation of historical data to prevent context window overflow.

Technical Specifications:

1. **SoP Truncation:** It must parse SoP.md and keep only the most recent 30 entries. Older entries must be moved to storage/archives/root/.
2. **Handoff Cleanup:** It must delete artifacts in storage/handoff/ older than 7 days, unless pinned to the current Dispatch Packet ID.

Forensic Insight: ops/bin/prune enforces the "**Sliding Window**" of history. It ensures that the SoP.md remains a "living ledger" rather than a museum, keeping the token count efficient for the AI.[1]

5.6 ops/bin/help: The Explanation System

- **Location:** ops/bin/help
- **Classification:** Operational Binary (Documentation Interface)
- **Purpose:** Provides a Context-Aware documentation search for the Operator.

Technical Specifications:

1. **Search Mechanism:** It must grep the docs/ directory for keywords, providing line numbers and file paths.
2. **Robustness:** The user prompt explicitly requested: "**UPDATE/IMPROVE/STRENGTHEN AND MAKE MORE ROBUST EXPLANATION SYSTEMS LIKE HELP.**" To satisfy this, the binary must be updated to specifically index and surface the new docs/ops/specs/ directory. When an operator types ./ops/bin/help dump, it should prioritize the technical spec docs/ops/specs/bin-dump.md over general mentions.
3. **Strategic Placement:** References to ops/bin/help must be placed in TASK.md and docs/library/MANUAL.md to ensure the Operator knows how to access this explanation system.

---

6. The Sheriff: Mechanistic Enforcement and The Skills Gap

The "Sheriff" layer consists of the linting tools in tools/lint/. A critical gap was identified in the tools/lint/project.sh script regarding the validation of **Skills**.

6.1 tools/lint/project.sh: The Pointer Validator (Expanded)

- **Current State:** Validates STELA.md files against docs/library/INDEX.md for Agents and Tasks.
- **The Gap:** It fails to validate **Skills** (S-LEARN-*). A project could reference a non-existent skill, leading to hallucinated workflows.
- **The Fix:** Update the script to scan for S-LEARN-* tokens and validate them against the

library index.
- **Mechanism:** "It scans project definition files to ensure they reference valid Agents (R-AGENT-*) and Tasks (B-TASK-*) **and Skills (S-LEARN-*)** that actually exist in the library index. Forensic Insight: This prevents projects from referencing deprecated or non-existent governance structures".[1]

6.2 tools/lint/truth.sh: The Linguistic Police

- **Purpose:** Enforces the Policy of Truth standards.
- **Mechanism:** Greps for forbidden typos of "Stela" and forbidden tokens (contractions like "don't", "can't") across strictly controlled surfaces.
- **Forensic Insight:** This is the primary defense against linguistic steganography. By enforcing "No Contractions," it maintains the rhythmic precision required for technical auditing.

6.3 tools/lint/context.sh: The Anti-Hallucination Guard

- **Purpose:** Verifies that every file listed in ops/lib/manifests/CONTEXT.md actually exists.
- **Mechanism:** Parses the manifest and checks for file existence. It enforces "Negative Constraints," failing if forbidden directories (like docs/library/skills) appear in the global manifest.

---

7. Operational Directive: DP-OPS-0028

Based on the forensic analysis, the "Deep Research" imperative, and the identification of the llms binary output issue and the project.sh validation gap, the following Dispatch Packet (DP-OPS-0028) is constructed.

This DP performs the following critical actions:

1. **Ratification:** Creates docs/library/FORENSICS.md to canonize this "Forensic Architecture Report."
2. **Correction:** Updates ops/bin/llms to enforce **Root Drop** (Flat Truth), reversing the previous instruction to hide artifacts.
3. **Hardening:** Updates tools/lint/project.sh to validate S-LEARN-* references.
4. **Documentation:** Populates docs/ops/specs/ with the technical specifications defined in Section 5.
5. **Explanation:** Strengthens ops/bin/help to surface these new specs.

DP-OPS-0028: Mechanistic Jurisdiction Hardening & Forensic Ratification

3.1 Freshness Gate (Must Pass Before Work)

Base Branch: main

Required Work Branch: work/forensic-ratification-2026-02-06

Base HEAD: d394111dd (preferred) or Not provided / Current (draft; lock at merge)

Required local re-check (worker runs; paste results in RESULTS):

- git rev-parse --abbrev-ref HEAD
- git rev-parse HEAD
- git status --porcelain

Worker must already be on the **Required Work Branch** listed above (no branch creation or switching).

STOP if any mismatch.

STOP if the Required Work Branch is missing.

STOP if told to create or switch branches.

3.2 Required Context Load (Read Before Doing Anything)

Worker must confirm loaded before acting:

- PoT, SoP, CONTEXT, MAP
- Plus any DP-scoped files listed here:
  - ops/bin/llms
  - tools/lint/project.sh
  - ops/bin/help

(OPEN is for Integrator/DP Writer state refresh; Worker does not read OPEN).

DISPOSABLE ARTIFACTS (chat logs, etc.) MUST NOT BE REFERENCED OR INCLUDED.

3.3 Scope and Safety

- **Objective:** Ratify forensic findings from the malicious model infection, harden the repository by establishing the Forensic Autopsy as a canonical reference, enforce Flat Truth architecture for context bundles, and expand operational documentation.
- **Non-Goals (optional, drift-killer):**
  - No changes to ops/bin/project logic (already handled in DP-OPS-0027).

    - No changes to PoT.md (already consolidated in DP-OPS-0025).

Target Files allowlist (hard gate)

- docs/library/FORENSICS.md (new)
- docs/library/INDEX.md
- ops/bin/llms
- tools/lint/project.sh
- docs/ops/README.md
- docs/ops/specs/bin-dump.md (new)
- docs/ops/specs/bin-help.md (new)
- docs/ops/specs/bin-llms.md (new)
- docs/ops/specs/bin-open.md (new)
- docs/ops/specs/bin-project.md (new)
- docs/ops/specs/bin-prune.md (new)
- ops/bin/help
- SoP.md

Allowlist rule: exact paths only; use (new) prefix only when the DP explicitly allows new files.

- **Conditional Requirements:** If canon/governance surfaces (e.g., TASK.md) are modified, SoP.md MUST be included in the allowlist and updated.
- **Stop Condition (scope):** If any required change falls outside the allowlist, **STOP** and report.
- **Forbidden (unless DP explicitly overrides):**
    - No edits to .github/
    - No edits to projects/ unless explicitly authorized
    - No renames/moves/deletes unless explicitly authorized
- **Worker Git Authority (non-negotiable):**
    - Workers do not create/switch branches.
    - Workers do not stage/commit/merge/push.
    - Workers only return diffs/results/receipt.
- **Precedence:** If any conflict exists, **PoT.md wins**. If unclear, STOP and ask.
- **Strict Stop Conditions (generic):**
    - STOP if any placeholder text remains in final outputs ([...], TODO, TBD, ...).
    - STOP if required DP inputs are missing (do not guess).
    - STOP if IDs / naming constraints in the DP are inconsistent (when applicable).

3.4 Execution Plan (A-E Canon)

3.4.1 State

- **Context:** The platform has recovered from a "malicious model infection" via DP-OPS-0027. The deep technical reasoning for the "Flat Truth" architecture is currently

missing from the canonical documentation. Previous drafts incorrectly suggested moving context bundles to storage; however, the "Flat Truth" architecture mandates that llms.txt bundles remain at the repository root to ensure a single, visible source of truth, even if this impacts porcelain cleanliness.

- **Drift:** Knowledge gap regarding SnatchML and DiSim failure; Operational drift in llms artifact placement logic; Missing technical specifications for operational binaries; tools/lint/project.sh lacks Skills validation.
- **Desired State:** The Forensic Autopsy is codified as docs/library/FORENSICS.md. The llms bundles are explicitly enforced at the repository root. docs/ops/ is populated with rigorous technical specifications. tools/lint/project.sh validates Skills references.

3.4.2 Request

Numbered tasks (concrete, no vibes):

1. **Create docs/library/FORENSICS.md:**
    - Content: Insert the full text of the "Forensic Architecture Report" provided in the attached PDF draft.
    - **Correction:** Modify Section 5.1.3 (Context Bundler) of the report text to reflect the Operator's override: "Operational Requirement: The system must enforce that these artifacts are dropped in the repository root. This ensures 'Flat Truth' visibility, prioritizing truth accessibility over 'Clean Working Tree' aesthetics."
    - Formatting: Ensure strictly No Contractions. Use standard Stela markdown headers.
2. **Update docs/library/INDEX.md:**
    - Register the new artifact under a new section "8. Forensic & Security Reports".
    - Key: forensic-autopsy
    - Title: Report: Forensic Autopsy
    - Path: docs/library/FORENSICS.md
3. **Update ops/bin/llms:**
    - Enforce Root Drop: Ensure OUT_DIR defaults to "${REPO_ROOT}".
    - Remove any logic that attempts to redirect default output to storage/handoff/.
    - Rationale: "Flat Truth" architecture requires context bundles (llms.txt, llms-full.txt) to exist at the root level to prevent the existence of "Two Truths" (one hidden, one visible).
4. **Update tools/lint/project.sh:**
    - Add Skills Validation: Update the script to scan STELA.md files for S-LEARN-* tokens.
    - Mechanism: It scans project definition files to ensure they reference valid Agents (R-AGENT-*), Tasks (B-TASK-*)*, and Skills (S-LEARN-*) that actually exist in the library index.
    - Forensic Insight: This prevents projects from referencing deprecated or non-existent governance structures.
5. **Populate docs/ops/:**
    - Create a specs/ directory under docs/ops/.
    - Create technical specification files for all non-document type scripts/tools/binaries:

        - docs/ops/specs/bin-dump.md
        - docs/ops/specs/bin-help.md
        - docs/ops/specs/bin-llms.md
        - docs/ops/specs/bin-open.md
        - docs/ops/specs/bin-project.md
        - docs/ops/specs/bin-prune.md
    - Content Requirement: Each file must include "Technical Specifications," "Requirements," "Usage," and "Forensic Insight." Update docs/ops/README.md to index these new specs.
6. **Update ops/bin/help:**
    - Strengthen explanation systems.
    - thoughtful/strategically place references for the Operator to access help/explanation.
    - Ensure help can locate the new docs/ops/specs content.
7. **Update SoP.md:**
    - Log the ratification of the forensic report and the Flat Truth enforcement.

Required Content (optional; use when doc/governance changes demand exact text/format):

- In docs/library/FORENSICS.md, ensure the table "Attack Characteristic | Description | Stela Manifestation" is preserved.

3.4.3 Changelog

- **Log (human-readable, 1–6 bullets):**
    - Added docs/library/FORENSICS.md (Forensic Autopsy report) and registered it in docs/library/INDEX.md.
    - Updated ops/bin/llms to strictly enforce root-level artifact generation for Flat Truth compliance.
    - Updated tools/lint/project.sh to validate S-LEARN-* skill references.
    - Populated docs/ops/specs/ with technical specifications for all operational binaries.
    - Enhanced ops/bin/help for better spec discovery.
    - Updated SoP.md.

3.4.4 Patch / Diff

- **Format:** Unified diff (preferred) or anchored snippets.

3.4.5 Receipt (Required)

Verification (MUST RUN; or report NOT RUN + reason + risk)

- ./ops/bin/dump --scope=platform (or repo-correct equivalent when scope differs)
- If dump refiners are used, include this fallback line in the DP:
    - Fallback:./ops/bin/dump --scope=platform --format=chatgpt --out=auto --bundle
- **Zero-Byte Check:** Verify dump is not empty (test -s <path>).

- Context lint (use repo-canonical command):
  - Prefer: bash tools/lint/context.sh
  - If not found and required: **STOP** and ask (do not invent a substitute)
- Truth lint (when PoT or canon surfaces touched; use repo-canonical command):
  - Prefer: bash tools/lint/truth.sh
  - If not found and required: **STOP** and ask (do not invent a substitute)

Skill capture (required during normal DP processing)

- Workflow: run ops/lib/scripts/skill.sh harvest to auto-generate a draft with provenance and semantic collision checks.
- Review: inspect the draft in storage/handoff/ for heuristic accuracy (Hot Zone and High Churn). Refine Invocation guidance and Solution.
  Do not edit the Provenance block.
- Promote: run ops/lib/scripts/skill.sh promote <draft_path> or log "no new skill promoted" with a rationale in RESULTS.
- Legacy capture is forbidden. Do not manually write skill files; the provenance block is required.
- If a DP requires skill capture, the DP Target Files allowlist must include docs/library/SKILLS.md.
- Proof required in RESULTS:
  - Harvest-only: command invoked plus draft path and ls storage/handoff/ output.
- Promotion: command invoked plus grep -n "Promotion Packet:" docs/library/SKILLS.md and grep -n "docs/library/skills/S-LEARN-" docs/library/INDEX.md.

Artifact bundle (required directory)

- storage/handoff/

Proof bundle checklist (required; naming pattern over exact names)

- storage/handoff/DP-OPS--RESULTS.md (required)
- storage/handoff/OPEN-PORCELAIN-[work-branch]-[hash].txt (required)
- storage/handoff/OPEN-[work-branch]-[hash].txt (required)
- storage/dumps/dump-full-... (or platform scope as applicable)
- git diff --name-only (include in RESULTS)
- git diff --stat (include in RESULTS)
- NEXT: one single action (include in RESULTS)

MANDATE: No receipt/proof bundle = **AUTOMATIC DISAPPROVAL**.

Closeout routine receipts not being produced CONSISTENTLY INCLUDING PORCELAIN, DUMP AND OPEN is a failure condition.

Partial receipts (e.g., missing OPEN artifacts when a session was started) are considered

failures.

REQUIRED OUTPUT

MUST include a "Status" block at the top of RESULTS:

- Scope summary: (1 sentence validating the loop)
- Tracked change: (What was edited)
- Verification: (Output of proofs)

RESULTS file must include (minimum)

- Verification results (RUN/NOT RUN + reason + risk)
- Patch/diff proof bundle (status/diff list or direct diff)
- Receipt pointers (OPEN + PORCELAIN + DUMP outputs/paths)
- git diff --name-only output
- git diff --stat output
- NEXT: one single action

Storage artifact flow (local-only, untracked):

- OPEN and OPEN-PORCELAIN outputs land in storage/handoff/.
- Dump bundles land in storage/dumps/.
- Results receipts land in storage/handoff/DP-OPS-0000-RESULTS.md (pattern).
- Operator stores DP drafts under storage/dp/intake/ and moves or copies to storage/dp/processed/ after completion (or when reusing a previously executed DP).
- These are retention paths only; they are not worker prerequisites unless the DP explicitly says so.

---

4. Closeout (Mandatory)

Worker closeout duties (end of session)

- [ ] Produce/Update: storage/handoff/DP-OPS--RESULTS.md
- [ ] If canon/governance surfaces were changed, include **SoP.md** in scope and update it in the same PR slice.
- [ ] Run ops/bin/prune (hygiene).
- [ ] Run ops/bin/llms (refresh context bundles).
- [ ] Ensure proof bundle checklist is satisfied (above).
  (No bundle = DISAPPROVE.)
- [ ] Fill out the Mandatory Closing Block (see below).
- [ ] Append a **TASK Work Log** entry (below) capturing:
  - what changed (high level)
  - gates/verification outcome (pass/fail or not run + reason)
  - blockers (if any)
  - **NEXT:** one single action

Mandatory Closing Block (Varied Wording Required)

Varied Wording provision: Each entry must use meaningfully distinct wording;

copy or minor tense changes are not acceptable. Entry 4 must differ from Entry 1.

1. Primary Commit Header (plaintext)
2. Pull Request Title (plaintext)
3. Pull Request Description (markdown)
4. Final Squash Stub (plaintext) (Must differ from #1)
5. Extended Technical Manifest (plaintext)
6. Review Conversation Starter (markdown)

Integrator/operator closeout duties

- [ ] Review diff vs DP scope/forbidden zones
- [ ] Run repo-gates / required checks
- [ ] Commit/push/open PR/merge (operator-controlled)

4.1 Thread Transition (Reset / Archive Rule)

Purpose: Prevent TASK from becoming a museum while preserving continuity.

When a DP is **complete** (merged) or **ended** (canceled/superseded):

1. Add a final Work Log entry that starts with: THREAD END: and includes:
   - outcome (merged/canceled/superseded)
   - pointer to the DP RESULTS file
   - NEXT: the single action (often “start new DP” or “await operator”)
2. Reset sections **2. Active Context** and **3. Current Dispatch Packet** to the next DP.
3. Keep the Work Log, but separate threads with a visible divider line:
   - ---
   - THREAD START: DP-OPS-

(If the Work Log gets too long, it’s acceptable to keep only the most recent thread’s entries here and move older thread logs into the DP RESULTS file—only when explicitly directed.)

---

5. Work Log (Timestamped Continuity)

Rule: This is not a transcript. It’s the durable breadcrumb trail.

Expectation: Append a new timestamped entry on every DP closeout (no exceptions).

Each entry ends with: NEXT: <one single action>.

- ** — DP-OPS-: [short note]. Verification:. Blockers: [none/...]. NEXT: [single action]

8. Conclusion and Strategic Outlook

The transition to a Unitary Mechanistic Jurisdiction represents a critical maturity point for the Stela platform. The "malicious model infection" served as a stress test that exposed the inherent vulnerabilities of "partial strictness" in generative AI environments.

The forensic analysis proves that in a text-based control plane, ambiguity is not merely a stylistic flaw; it is a security vulnerability. By abolishing the "Multi-verse," enforcing a "Zero Trust" architecture, and "lobotomizing" core binaries into deterministic loaders, the platform has eliminated the shadow compliance zones that facilitated the attack.

The "Sheriff" layer now provides continuous, automated policing of the Policy of Truth, ensuring that the system remains in a state of autopoietic equilibrium. Furthermore, the operational correction regarding llms artifact placement highlights the extreme discipline required for this architecture. Even a benign action like generating a context bundle can become a vulnerability if it degrades the "**Visibility of Truth**" required for verification.

The mandates ratified in **DP-OPS-0028** ensure that Stela operates not just as a codebase, but as a provenance-forward, forensic-grade system capable of resisting the entropic drift inherent in AI-mediated development. The platform is now hardened, grounded, and prepared for future evolution under the singular authority of the Policy of Truth.[1]

Works cited

1. Stela Agent Constitution DP Draft.pdf
2. SnatchML: Hijacking ML models without Training Access - arXiv, accessed February 6, 2026, https://arxiv.org/html/2406.01708v2
3. Model for Peanuts: Hijacking ML Models without Training Access is, accessed February 6, 2026, https://aisecurity-portal.org/en/literature-database/model-for-peanuts-hijacking-ml-models-without-training-access-is-possible/
4. SnatchML: Hijacking ML Models Without Training Access | Request PDF - ResearchGate, accessed February 6, 2026, https://www.researchgate.net/publication/391998674_SnatchML_Hijacking_ML_Models_Without_Training_Access
5. Get a Model! Model Hijacking Attack Against Machine Learning Models - Semantic Scholar, accessed February 6, 2026, https://www.semanticscholar.org/paper/Get-a-Model!-Model-Hijacking-Attack-Against-Machine-Salem-Backes/6d0f8427064a7eaa04947a2ba4c91c98efa734da
6. Semantic Communication for the Internet of Everything: From Molecular to Space Networks - Hanlin Cai, accessed February 6, 2026, https://caihanlin.com/mypaper/thesis/MPhil_thesis.pdf
7. Sine: Similarity is Not Enough for Mitigating Local Model Poisoning Attacks in

Federated Learning | Request PDF - ResearchGate, accessed February 6, 2026, https://www.researchgate.net/publication/377372984_Sine_Similarity_is_not_enough_for_mitigating_Local_Model_Poisoning_Attacks_in_Federated_Learning
8. Graph Representation-based Model Poisoning on Federated Large Language Models, accessed February 6, 2026, https://arxiv.org/html/2507.01694v2
9. Graph Representation-based Model Poisoning on Federated Large Language Models - arXiv, accessed February 6, 2026, https://arxiv.org/pdf/2507.01694
10. Data Architecture 101, accessed February 6, 2026, https://aicouncil.com/hubfs/Data%20Council/sg%20slides/day%201/1.5%20Bence%20Faludi%2C%20Independent%20Consultant%20-%20Data%20Architecture%20101%20for%20Your%20Business%20-%20Bence%20Faludi.pdf